



Jobcoin Clock-in Program

Security Assessment (Summary Report)

March 5, 2026

Prepared for:

Marshall Weaver

Jobcoin

Prepared by: **Coriolan Pinhas**

Table of Contents

Table of Contents	1
Project Summary	2
Project Targets	3
Executive Summary	4
Summary of Findings	5
Detailed Findings	6
1. Clock-in denial of service via pre-created ATA	6
2. Stale token balance is logged after CPI and after closing the account, which can mislead monitoring and incident response	8
3. Early-withdrawal penalty rounds down, enabling penalty minimization via small splits	10
A. Vulnerability Categories	11
B. Code Quality Issues	13
C. Fix Review Results	13
Detailed Fix Review Results	15
D. Fix Review Status Categories	16
About Trail of Bits	17
Notices and Remarks	18

Project Summary

Contact Information

The following project manager was associated with this project:

Tara Goodwin-Ruffus, Project Manager
tara.goodwin-ruffus@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Coriolan Pinhas, Consultant
coriolan.pinhas@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
January 15, 2026	Pre-project kickoff call
January 23, 2026	Delivery of report draft
January 26, 2026	Report readout meeting
February 23, 2026	Delivery of final comprehensive report
March 5, 2026	Delivery of final comprehensive report with fix review

Project Targets

The engagement involved reviewing the following target.

Jobcoin Clock-in Program

Repository	https://github.com/Wisteria-Co/jobcoin-clockin-contract
Version	665a260428149bc96e2e9531f9b4f4ddc628d2e0
Type	Rust
Platform	Solana

Executive Summary

Engagement Overview

Jobcoin engaged Trail of Bits to review the security of its **clock-in program**.

The clock-in program is an Anchor Solana program that lets users lock Jobcoin SPL tokens for a set time and later withdraw them. Each lock uses a PDA (from the user's pubkey and an index) that stores the lock info and controls a PDA-owned ATA. Early withdrawals send 30% to Jobcoin Labs, while mature withdrawals return 100%.

A team of one consultant conducted the review from January 20 to 22, 2026, for a total of three engineer-days of effort. With full access to source code and documentation, we performed static and dynamic testing of the clock-in program, using automated and manual processes.

Observations and Impact

The review focused on the on-chain logic for `clock_in/clock_out`, PDA derivations, ATA creation, CPI transfers/close behavior, penalty math, and logging pathways. Coverage included user flows (deposit, early withdrawal, mature withdrawal) and the correctness of state transitions and emitted logs.

This was a time-boxed engagement (three engineer-days) and did not include off-chain components, which are critical to the system's overall security and correctness.

We identified three findings: one low-severity auditing/logging issue and two informational data-validation issues. The most impactful risk is a denial-of-service vector that can prevent a user from using a specific `clock_in_index` by pre-creating the PDA's ATA with dust. The logging issue does not affect on-chain state but can mislead off-chain monitoring and incident response if logs are treated as authoritative. The early-withdrawal penalty rounding behavior can be minimized via split deposits. This is primarily a protocol-economics concern rather than an immediate security break.

Recommendations

Prioritize adding comprehensive end-to-end tests to cover all user flows and edge cases (including early withdrawal, mature withdrawal, pre-created ATAs, and account close paths) to ensure strong, scenario-level coverage. In parallel, ensure off-chain components that calculate user rewards or drive operational decisions do not rely on potentially stale or misleading program logs. Addressing these two areas should be treated as the primary concern, as they materially reduce the risk of incorrect rewards and operational misinterpretation.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Clock-in denial of service via pre-created ATA	Data Validation	Informational
2	Stale token balance is logged after CPI and after closing the account, which can mislead monitoring and incident response	Auditing and Logging	Low
3	Early-withdrawal penalty rounds down, enabling penalty minimization via small splits	Data Validation	Informational

Detailed Findings

1. Clock-in denial of service via pre-created ATA

Severity: Informational

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-JOBCOIN-1

Target: programs/jobcoin-clock-in-program/src/lib.rs

Description

The `clock_in` instruction uses `init_if_needed` for the `pda_token_account` and requires that the account balance be zero, as shown in figure 1.1.

```
require_eq!(ctx.accounts.pda_token_account.amount, 0,
  ErrorCode::AlreadyClockedIn); //I believe this will always be caught by Anchor's
init in account initialization.
```

Figure 1.1: The program revert if the balance is not equal to zero

jobcoin-clockin-contract/programs/jobcoin-clock-in-program/src/lib.rs#L33-L33

Because the PDA address is deterministic (derived from `user.key()` and `clock_in_index`), and the associated token account address is also deterministic (derived from the PDA authority and token mint), an attacker can front-run and create the ATA for any user's future clock-in attempt and send dust (1 token) to it.

When the legitimate user attempts to `clock_in` with that index, the `init_if_needed` will succeed (account already exists), but the `amount == 0` check will fail, permanently preventing that user from using that specific `clock_in_index`.

Exploit Scenario

1. Alice plans to call `clock_in` with `clock_in_index = 0`.
2. Bob (attacker) computes Alice's PDA address: `PDA = derive(["user", alice.pubkey, index=0])`.
3. Bob computes the ATA address: `ATA = get_associated_token_address(PDA, JOBCOIN_MINT)`.
4. Bob creates the ATA (pays rent) and sends 1 lamport worth of tokens to it.

5. Alice attempts `clock_in(amount=1000, index=0, period=2)`.
6. The instruction fails with `ErrorCode::AlreadyClockedIn` because `pda_token_account.amount = 1 ≠ 0`.
7. Alice is prevented from using `clock_in_index = 0`.

Recommendations

Short term, replace `init_if_needed` by `init` and remove `require_eq!(ctx.accounts.pda_token_account.amount, 0, ErrorCode::AlreadyClockedIn)`.

Long term, add a unit test that pre-creates the ATA with dust and confirms `clock_in/clock_out` still succeeds.

2. Stale token balance is logged after CPI and after closing the account, which can mislead monitoring and incident response

Severity: Low

Difficulty: Low

Type: Auditing and Logging

Finding ID: TOB-JOBCOIN-2

Target: programs/jobcoin-clock-in-program/src/lib.rs

Description

The program logs `pda_token_account.amount` after CPI calls and after closing the token account. This creates a logging correctness issue: Anchor deserializes account data at instruction entry, and it does not automatically reload `TokenAccount` data after a CPI call updates that account on-chain. As a result, `ctx.accounts.pda_token_account.amount` can still show a pre-CPI snapshot even when the SPL Token program already applied the transfer or the close.

This issue does not change on-chain token movement by itself, but it makes the program logs risky to use as an audit trail. For example, a deposit log can show `0` even though tokens moved into the account, and a close-path log can show a non-zero balance even though the token account is closed. If the off-chain process uses these logs to confirm actions or guide incident response, it can lead to the wrong action or delay triage.

```
msg!("Clock in complete! PDA token balance: {}",
ctx.accounts.pda_token_account.amount);
```

Figure 2.1: Logging a token balance from `ctx.accounts.pda_token_account` after a CPI transfer

[\(jobcoin-clockin-contract/programs/jobcoin-clock-in-program/src/lib.rs#L55\)](#)

```
msg!("Clock out complete! PDA token balance: {}",
ctx.accounts.pda_token_account.amount);
```

Figure 2.1: Logging a token balance from `ctx.accounts.pda_token_account` after closing the token account

[\(jobcoin-clockin-contract/programs/jobcoin-clock-in-program/src/lib.rs#L113\)](#)

Exploit Scenario

Alice runs an off-chain monitor that reads program logs to confirm deposits and withdrawals, to trigger alerts, and to support incident response. Eve triggers normal `clock_in` and `clock_out` activity, and Alice's monitor reads a stale

`pda_token_account.amount` value that does not match on-chain state after the CPI transfer or after the close. Alice treats the log as correct, and it makes the monitor raise false alerts or miss real issues, slowing response and making investigations harder.

Recommendations

Short term, reload the token account before logging post-CPI fields (or log only deterministic values like `amount / amount_to_transfer_to_user` that do not require reading refreshed on-chain state).

Long term, add tests that assert emitted logs match expected post-instruction outcomes for deposit/withdrawal flows (including the close-account path) to prevent regressions.

3. Early-withdrawal penalty rounds down, enabling penalty minimization via small splits

Severity: Informational	Difficulty: High
Type: Data Validation	Finding ID: TOB-JOBCOIN-3
Target: programs/jobcoin-clock-in-program/src/lib.rs	

Description

The early-withdrawal penalty is calculated with integer division and is therefore rounded down, which under-charges the intended 30% penalty for any amount that is not a multiple of 100 (and yields zero penalty for very small balances). This allows users to minimize or eliminate penalties by splitting their deposits across many small clock-ins and withdrawing early from each.

```
// Early withdrawal penalty: 30%  
let penalty = ((ctx.accounts.pda_token_account.amount as u128 * 30) / 100) as u64;
```

*Figure 3.1: Penalty uses integer division, rounding down to the nearest token unit
([programs/jobcoin-clock-in-program/src/lib.rs#L80-L81](#))*

Because integer division truncates, any amount < 4 (in base units) yields a penalty of 0. More generally, the effective penalty is always $\leq 30\%$, never strictly 30%, and can be made arbitrarily low by splitting into sufficiently small deposits if the token has divisible units.

Recommendations

Short term, round the penalty up instead of down, or enforce a minimum penalty of 1 base unit for any non-zero early withdrawal.

Long term, ensure that the code always rounds in favor of the protocol.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Issues

This appendix contains findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Misleading log label for `clock_in_period`.** The `clock_in` function logs `clock_in_period` but labels it as `clock_in_index`, which makes debugging and telemetry confusing. This can mislead anyone reading logs or tracing issues. Update the message string to match the variable name.
- **Undocumented vesting period allows 10-second locks.** `DURATIONS` includes an entry of 10 seconds with no documentation or mention in the comments. This creates an undocumented vesting period that changes program behavior in non-obvious ways. Document it clearly to improve the program's understandability.
- **`user_token_account.owner == user.key()` blocks delegate usage.** The `account constraint` enforces that the token account owner is the user. That means delegated transfers (or multisig authority flows) are not supported, which might be an intentional design choice but should be documented.
- **TooSoon error code is defined but never used.** The error enum includes `TooSoon`, but no instruction ever returns it. This is misleading for maintainers and can hide missing logic (e.g., a minimum lock duration check). Either implement the check or remove the error to keep the code and error surface aligned.

C. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On February 23, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Jobcoin team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, Jobcoin has resolved all three issues described in the report. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Clock-in denial of service via pre-created ATA	Informational	Resolved
2	Stale token balance is logged after CPI and after closing the account, which can mislead monitoring and incident response	Low	Resolved
3	Early-withdrawal penalty rounds down, enabling penalty minimization via small splits	Informational	Resolved

Detailed Fix Review Results

TOB-JOBCOIN-1: Clock-in denial of service via pre-created ATA

Resolved in [PR #1](#). The 0 amount check has been removed to avoid denial-of-service attacks, while `init_if_needed` has been replaced by `init` to keep the original protection.

TOB-JOBCOIN-2: Stale token balance is logged after CPI and after closing the account, which can mislead monitoring and incident response

Resolved in [PR #1](#). The token account balance is reloaded before logging post-CPI fields in the first case. Deterministic values (`amount_to_transfer_to_user`, `penalty`) are logged in the second case.

TOB-JOBCOIN-3: Early-withdrawal penalty rounds down, enabling penalty minimization via small splits

Resolved in [PR #1](#). The early-withdrawal penalty calculation has been modified to round up.

D. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Jobcoin under the terms of the project statement of work and has been made public at Jobcoin's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.